Word Ladder II (hard)

Problem Statement

A `transformation sequence` is a sequence that starts from the word `beginWord`, ends with `endWord`, and contains a word from the given dictionary `wordList` in the middle i.e. a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Each adjacent pair of words differs by a single letter.
- Each s_i for `1 <= i <= k` is in wordList. sk == endWord
- The `beginWord` does not need to be in wordList.

Given two strings, `beginWord` and `endWord`, and a `wordList`, return all the `shortest transformation sequences`, or an empty list if no such sequence exists. Each sequence should be in the form of `[beginWord, s1, s2, ..., sk]`.

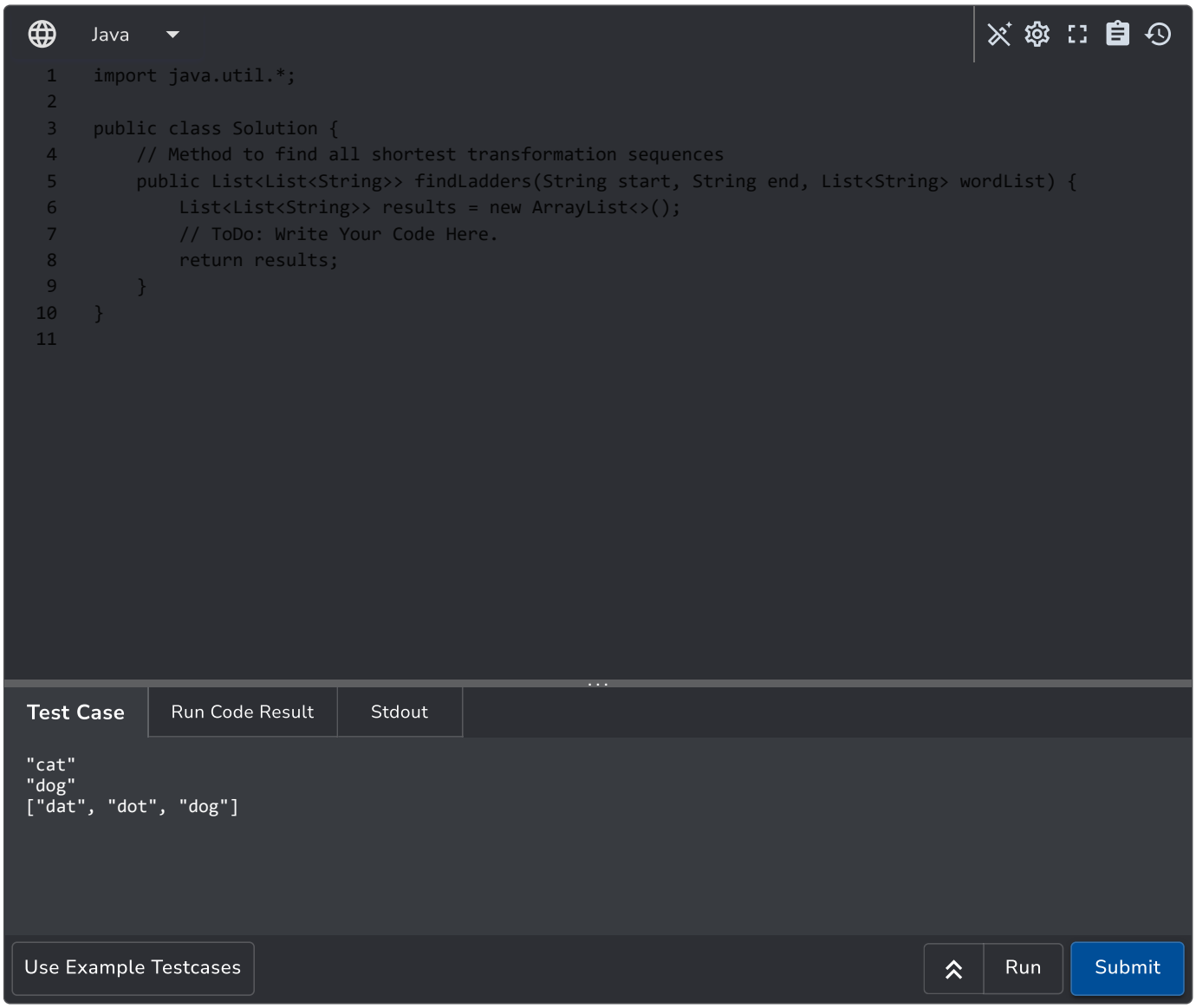
Examples

1. **Example 1**
   - **Input:** start = "cat", end = "dog", wordList = ["dat", "dot", "dog"]
   - **Expected Output:** [["cat", "dat", "dot", "dog"]]
   - **Justification:** The sequence "cat" -> "dat" -> "dot" -> "dog" changes one letter at a time, and each word exists in the wordList. It's the shortest path from "cat" to "dog".
2. **Example 2**
   - **Input:** start = "bit", end = "fog", wordList = ["but", "put", "big", "pot", "pog", "dog"]
   - **Expected Output:** []
   - **Justification:** Although there are words in the list that are one letter away from "bit", there's no sequence that successfully transforms "bit" to "fog" through words in the wordList.
3. **Example 3**
   - **Input:** start = "hot", end = "cog", wordList = ["dot", "dog", "lot", "log", "cog"]
   - **Expected Output:** [["hot", "dot", "dog", "cog"], ["hot", "lot", "log", "cog"]]

- **Justification:** This example shows two possible shortest transformation sequences from "hot" to "cog". Each sequence involves changing one letter at a time with all intermediate words existing in the given wordList.

Try it yourself

Try solving this question here:



← Previous

Solution: Word Ladder

Next →

Solution: Word Ladder II

Mark as Completed